# *UART APB Core*

## 1. General Description

The UART-APB core is a serial communication controller with a serial data interface that is intended primarily for embedded systems and designing ASIC. The UART-APB core can be used to interface directly to industry standard UARTs. The UART-APB core is intentionally a subset of full UART capability to make the function cost-effective in a programmable device.

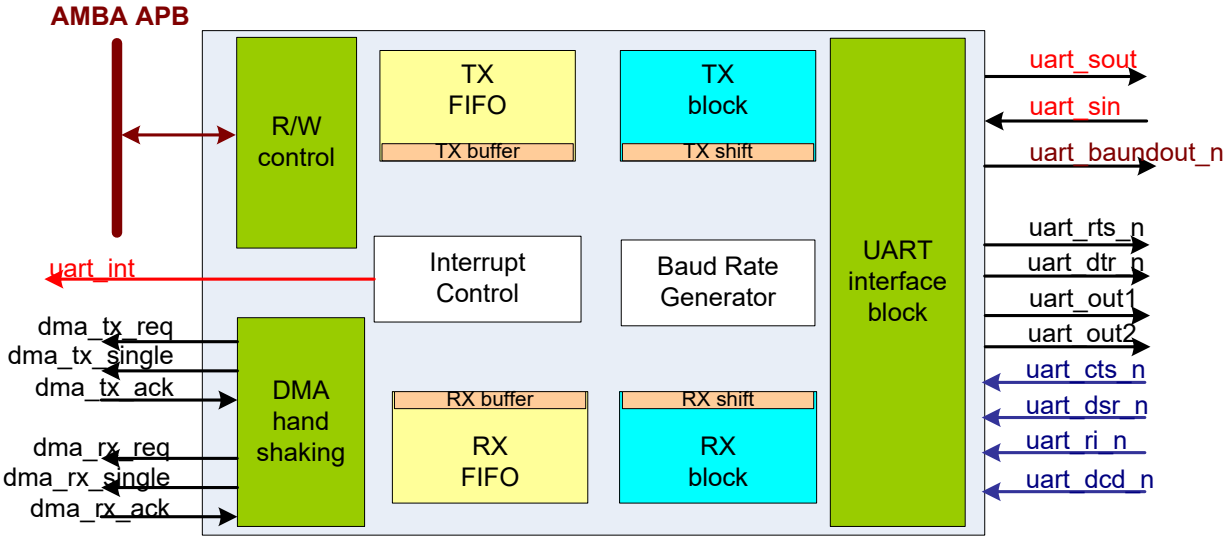


**Figure 1. UART APB block diagram**

### 1.1 Read/ Write Control

This module handles communication with the processor of the system throught AMBA APB. The processor controls the UART APB by managing all writing and reading of internal registers: status registers and control registers.

### 1.2 DMA Handshake

The UART APB supports DMA signalling with the use of two output signals (*uart_dma_tx_req_n* and *uart_dma_rx_req_n*) to indicate when data is ready to be read or when the transmit FIFO is empty. The UART APB uses two DMA channels, one for the transmit data and one for the receive data.

### 1.3 Interrupt

The assertion of the UART-APB interrupt output signal occurs whenever one of the several prioritized interrupt types are enabled and active. The following interrupt types can be enabled with the IER register:

- Receiver Error
- Receiver Data Available

- Character Timeout (in FIFO mode only)
- Transmitter Holding Register Empty at/below
- Modem Status
- Busy Detect Indication.

## 1.4 Modem

A modem *(**mo**dulator-**dem**odulator)* is a device that modulates an analog carrier signal to encode digital information, and also demodulates such a carrier signal to decode the transmitted information. The goal is to produce a signal that can be transmitted easily and decoded to reproduce the original digital data. Modems can be used over any means of transmitting analog signals, from driven diodes to radio.

## *1.5* Baud Rate Generator

Baud Rate Generator Takes the input clock (PCLK), and divides it by a programmed value (from 1 to $2^{16}$– 1). This divided clock is then divided by 16 to create the baudrate for the internal transmitter and internal receiver.

## 1.6 Receiver

The receiver handles the receiving of the serial input data. It is programmable to recognize data widths such as 5, 6, 7 or 8 bits, various parity settings. It checks for errors in the input data stream, and if the input data doesn't have problems it is placed either in the Receiver Holding register or in the receive-FIFO depending on the mode programmed.

## 1.7 Transmitter

The transmitter handles the transmission of data, written to the Transmitter Holding register (or transmit-FIFO). The transmitter adds start, parity and stop bits to the data which are transmitted so that the receiving device can do the proper error handling and receiving.

## 1.8 FIFOs

The FIFOs can be selected to be either external RAM or internal D-flip-flop. When external RAM support is chosen, both synchronous and asynchronous read-port memories are supported.

The UART APB can be configed:

1. Non-FIFO
2. Internal FIFO
3. External FIFO

## 2. Feature

- AMBA APB protocol, version 2.0
- Configurable parameters for the following:
  - ✓ APB data bus widths of 8, 16 and 32
  - ✓ DMA interface signal polarity
  - ✓ Transmit and receive FIFO depths of none, 16, 32, 64,…,2048
  - ✓ Internal or external FIFO (RAM) selection
  - ✓ Shadow registers to reduce software overhead and also include a software programmable reset
  - ✓ Auto Flow Control mode as specified in the ***16750*** standard
  - ✓ Loopback mode that enables greater testing of Modem Control and Auto Flow Control
  - ✓ Transmitter Holding Register Empty (THRE) interrupt mode
  - ✓ Busy functionality
- Functionality based on the ***16550*** industry standard, as follows:
  - ✓ 5, 6, 7 or 8 data bits
  - ✓ 1, 1.5 or 2 stop bits
  - ✓ Odd, even, space, mark, or no parity
  - ✓ Line break generation and detection
  - ✓ DMA signaling with two programmable modes
  - ✓ Prioritized interrupt identification
- Programmable FIFO enable/disable
- Programmable baud rate generator
- Parity, framing and overflow error detection
- Line break generation and detection
- Modem and status lines are independently controlled
- Local loopback mode
- Full prioritized interrupt system
- Controls Modem control signals:CTS, RTS, DSR, DTR, RI and DCD

# 3. Functional Description

## 3.1 UART (RS232) Serial Protocol

The data frame for transmission is programmable. It consists of a start bit, 5 to 8 data bits, an optional parity bit and 1 to 2 stop bits, which can be specified by the line control register *(LCR)*. The transmitter can also produce the break condition. The break condition forces the serial output to logic 0 state for one frame transmission time.

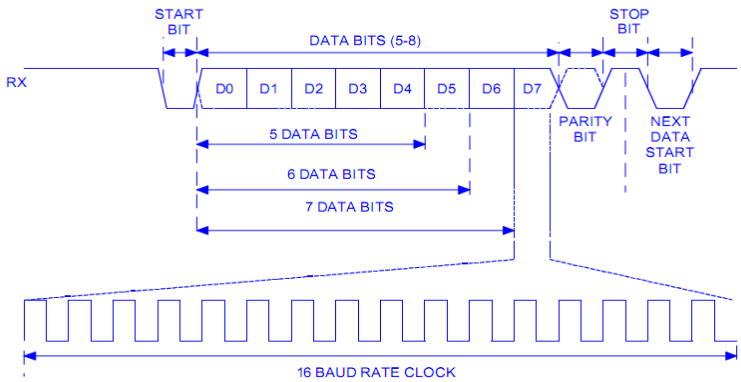


**Figure 2. Serial data format**

All the bits in the transmission are transmitted for exactly the same time duration. This is called as a BIT time. A bit time equals 16 baud clocks. The receiver samples serial input at the mid point of the BIT time. The flow figure shows the sampling points of the first couple of bits in a serial character.

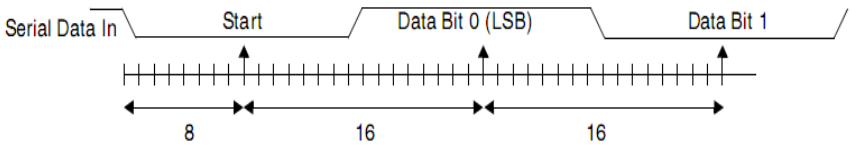


**Figure 3. Receiver serial data sample points**

The baud rate of the DW_apb_uart is controlled by pclk and the Divisor Latch Register (DLH and DLL). The flow figure shows the timing diagram for the uart_baudout_n output for different divisor values.

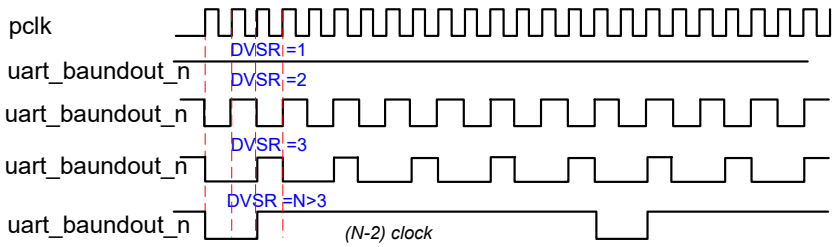


**Figure 4.Baud Clock Reference Timing Diagram**

Uart_baudout_n signal is only included when BAUD_CLK_USER is selected.

## 3.2 FIFO Support

The UART APB can be configured to implement FIFOs.

- Non FIFO
- Internal FIFO (registers)
- External FIFO (SRAM)

When Non FIFOs are implemented, there is only a single receive data byte and transmit data byte can be stored in the *RBR* and *THR.*

In FIFO mode, the FIFOs can be selected to be either external FIFO (SRAM) or internal FIFO (registers). If the configured depth is large, the FIFO should be selected to be external FIFO. When external RAM support is chosen, either synchronous or asynchronous RAMs can be used. The flow figure shows the timing diagram for both asynchronous and synchronous RAMs

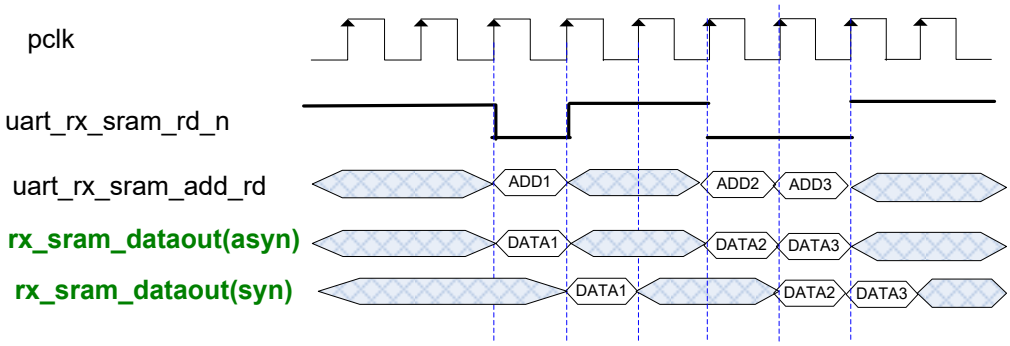


**Figure 5. Timing for RAM reads**

Similarly, you can use synchronous RAM for writes, which registers the data at the current address out. The flow figure shows the timing diagram for RAM writes.

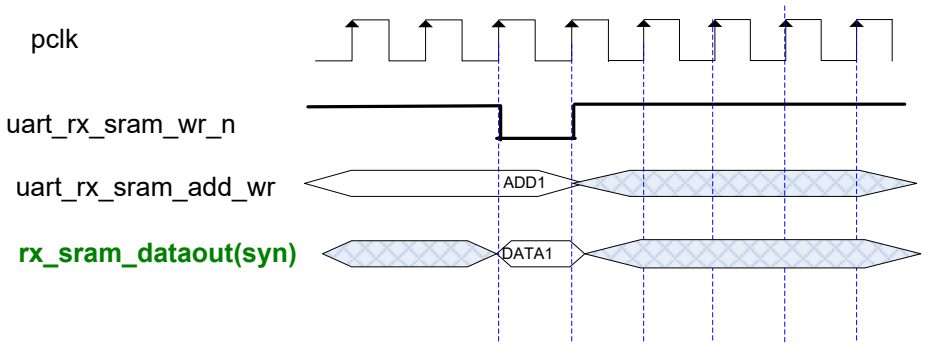


**Figure 6. Timing for RAM writes**

**Note:**

These timing diagrams assume the RAM used has a chip select port that is tied to an active value.

## 3.3 Interrupts

The UART APB has six types of interrupts:

- Receiver Error
- Receiver Data Available
- Character Timeout (in FIFO mode only)
- Transmitter Holding Register Empty at/below threshold (in Programmable THRE interrupt mode)
- Modem Status
- Busy Detect Indication

### 3.3.1 Receiver Error

The UART APB interrupt output signal occurs whenever one of the several prioritized interrupt types are enabled and active. When an interrupt occurs, the master should read the *IIR* to determine the source of the interrupt before dealing with it accordingly.

The receiver can detect overrun error, parity error, frame error and break condition, each of which will set an interrupt flag if it is enabled (*IER [2]*).

1. The overrun error indicates that new data has overwritten the old data before the old data has been read.
2. The parity error indicates that the receiver has detected an unexpected parity condition.
3. The frame error indicates that the received data does not have a valid stop bit.
4. The break condition indicates that the receiver input is held in the logic 0 state for a duration longer than one frame transmission time.

When the FIFOs are selected, the UART APB has the status FIFO besides the receive-FIFO. The status FIFO indicates an error status of data. The error interrupt will be issued only when the data, which has an error, is ready to read out. To clear the status FIFO, the *RBR* with an error or *LSR* must be read out.

***For example:***

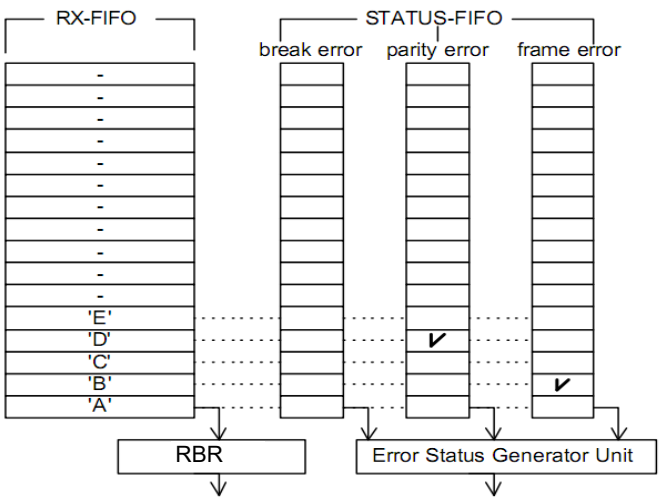


**Figure 7. a case showing uart receiving 5 characters with 2 errors**

**Table 1: reading data from Rx FIFO**

| Time | Sequence flow | Error interrupt |
|---|---|---|
| #0 | When no character is read out | - |
| #1 | After A is read out | The frame error(in B) interrupt occurs |
| #2 | After B is read out | - |
| #3 | After C is read out | The parity error(in D) interrupt occurs |
| #4 | After D is read out | - |

### 3.3.2 Receiver Data Available

- When the receiver transfers the data of the receive shifter to the receive FIFO and the number of received data reaches receiver FIFO Trigger Level. That can be a cause of the interrupt if the receiver interrupt it is enable *(IER [0])*.
- In the Non-FIFO mode, transfering the data of the receive shifter to the receive FIFO. That can be a cause of the interrupt if the receiver interrupt it is enable *(IER [0])*.

### 3.3.3 Character Timeout

No characters in or out of the receiver FIFO during the last 4 character times and there is at least 1 character in it during this time. Timeout can be a cause of the interrupt if the receiver interrupt it is enable *(IER [0])*.

### 3.3.4 Transmitter Holding Register Empty at/below threshold

- When the transmitter data from its transmitter FIFO to its transmit shifter and the number of data left in transmitter FIFO reaches transmitter FIFO trigger level, That can be a cause of the interrupt if the transmitter interrupt it is enable *(IER [1])*, and a Programmable THRE Interrupt is configured (see 3.4).
- In the Non-FIFO mode, transfering data from the transmit FIFO to the transmitter shifter can be a cause of the interrupt if the transmitter interrupt it is enable *(IER [1])*.

### 3.3.5 Modem Status

- When there is at least 1 bit of *MSR [3:0]* is active (Delta Data Carrier Detect, Trailing Edge of Ring Indicator, Delta Data Set Ready, and Delta Clear to Send). That can be a cause of the interrupt if the receiver interrupt is enable*(IER [3])*.

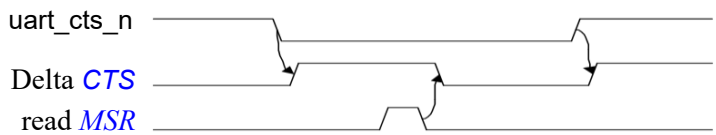


**Figure 8. *uart_cts_n* and *delta CTS* timing**

### 3.3.6 Busy Detect Indication

- Writing data into the *LCR* register during a transaction. That can be a cause of the interrupt.
- This mode only is valid when a Busy Detection (configed parameter) is selected.
- Busy functionality helps to safe guard against errors if the *LCR* register is changed during a transaction. So, they should only be set during initialization.

## 3.4 Programmable THRE Interrupt

The UART APB can be configured to have a Programmable THRE Interrupt. If FIFO mode or THRE mode is not implemented, this mode is selected. To reduce the overall gate counts, none of the logic is implemented if the mode is not selected.

The THRE Interrupt is like as *uart_dma_tx_req*. It is active at and below a programmed transmitter FIFO empty threshold level. This threshold level is programmed into *FCR[5:4]*. The available empty thresholds are: empty, 2-byte, ¼-FIFO, and ½-FIFO *(See "FCR").*

The THRE Interrupt is a cause of the interrupt if it is enable *(see IER)*. The uart_dma_tx_req is set to high to request DMA that writing data to the transmit-FIFO.

## 3.5 Auto Flow Control

### 3.5.1 Auto Flow operation

The UART APB can be configured to have an autoflow control mode. If the FIFO mode is not implemented, the autoflow mode is not selected. If the autoflow control mode is enabled, it is a programmable mode (*MCR[5]*).

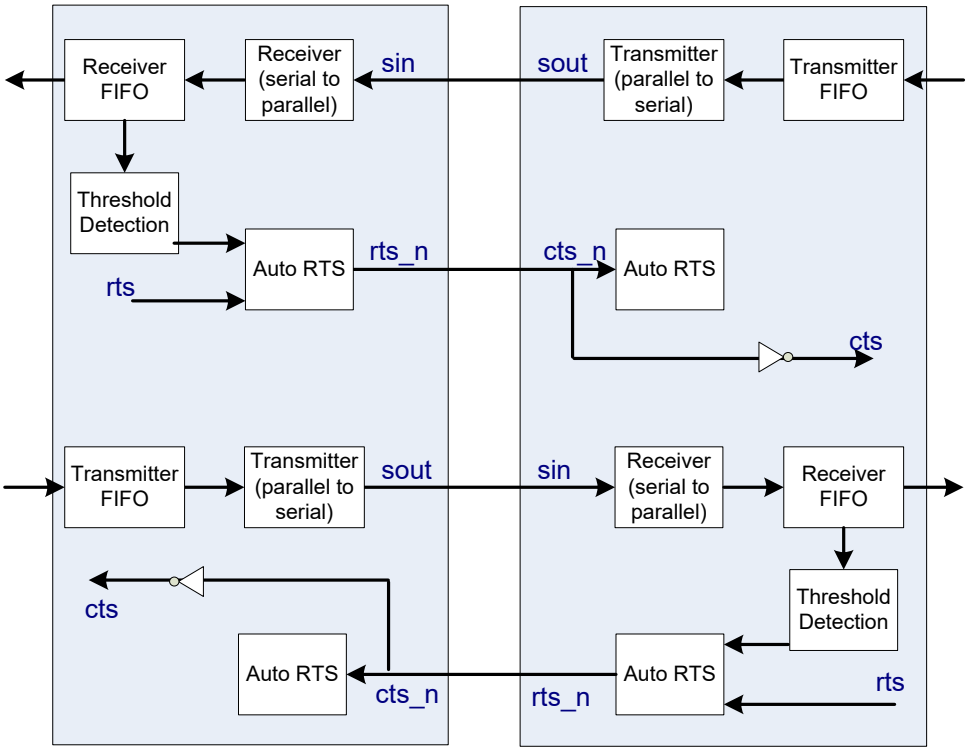


**Figure 9. Auto flow control block diagram**

Autoflow control comprises auto-CTS and auto-RTS. With auto-CTS, the CTS input must be active before the transmitter FIFO can emit data. With auto-RTS, RTS becomes active when the receiver needs more data and notifies the sending serial device. When RTS is connected to CTS, data transmission does not occur unless the receiver FIFO has space for the data.

Auto RTS and Auto CTS are described as follows:

### 3.5.2 Auto RTS

- Auto RTS – Becomes active when the following occurs:
  - Auto Flow Control is selected during configuration.
  - FIFOs are implemented.
  - RTS *(MCR[1]* bit and *MCR[5]* bit are both set).
  - FIFOs are enabled (*FCR[0])* bit is set).
  - The receiver FIFO level reaches the threshold set by *FCR [7:6]*

When the *uart_rts_n* is low (active), then the *uart_rts_n* output is forced inactive (high) when the receiver FIFO has available space (until it is completely empty).

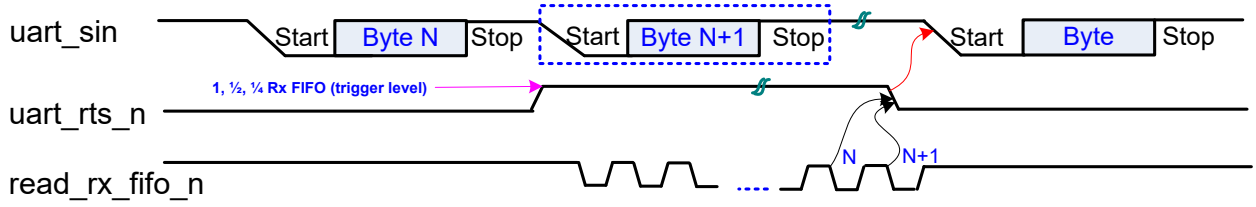


**Figure 10. Auto RTS timing**

If the trigger level is 2 less than full, RTS is deasserted when the receiver FIFO is almost full and the receiver receives the first data bit of the next byte.

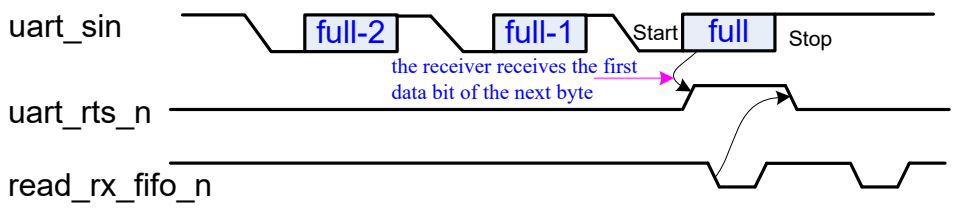


**Figure 11. Auto RTS timing**

### 3.5.3 Auto CTS

- Auto CTS – becomes active when the following occurs:
  - Auto Flow Control is selected during configuration
  - FIFOs are implemented
  - AFCE (*MCR[5]* bit is set)
  - FIFOs are enabled through FIFO Control Register *FCR[0]* bit

When Auto CTS is enabled (active), the transmitter is disabled whenever the *uart_cts_n* input becomes inactive (high). This prevents overflowing the FIFO of the receiving UART.

**Note:**

1. If the *uart_rts_n* goes high **before** the middle of the last stop bit of the current byte, the transmitter finishes sending the current byte, and it does not send the next byte.
2. The two above figures, the *uart_rts_n* goes high **after** the middle of the last stop bit of the current byte.

## 3.6 DMA Support

The UART APB supports DMA signalling with the use of two output signals (*uart_dma_tx_req_n* and *uart_dma_rx_req_n*) to indicate when data is ready to be read or when the transmit FIFO is empty. The UART APB uses two DMA channels, one for the transmit data and one for the receive data.

There are two DMA modes: mode 0 and mode 1, controllable via bit 3 of the FIFO Control Register (only DMA mode 0 is available when the FIFOs are not implemented or disabled).

### 3.6.1 Burst Transaction

#### 3.6.1.1 DMA mode 0 support

1. The uart_dma_tx_req
   - The *uart_dma_tx_req* signal becomes active when the following occurs:
     - When the *THR* is empty in non-FIFO mode
     - When the transmitter FIFO is empty in FIFO mode with Programmable THRE interrupt mode disabled
     - When the transmitter FIFO is at or below the programmed threshold with Programmable THRE interrupt mode enabled
   - The *uart_dma_tx_req* goes inactive under the following conditions:
     - When *THR* is written into the Transmitter Holding Register or transmitter FIFO with Programmable THRE interrupt mode disabled
     - When the transmitter FIFO is above the threshold with Programmable THRE interrupt mode enabled.

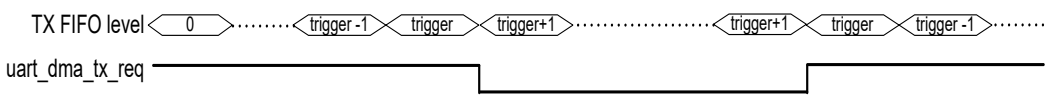


**Figure 12. *uart_dma_tx_req* timing (DMA mode 0)**

2. The uart_dma_rx_req
   - The *uart_dma_rx_req* signal becomes active when the following occurs:
     - When the *RBR* is available in non-FIFO mode.
     - When the receiver FIFO is at or above the full trigger level in FIFO mode.
   - The *uart_dma_rx_req* goes inactive under the following conditions:

- When the *RBR* is read in non-FIFO mode.
- When the receiver FIFO is below the full trigger level in FIFO mode.

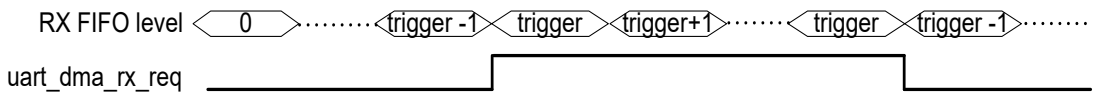


**Figure 13. *uart_dma_rx_req* timing (DMA mode 0)**

### 3.6.1.2 DMA mode 1 support

1. The uart_dma_tx_req.
   - ➢ The *uart_dma_tx_req* signal becomes active when the following occurs:
     - When the *THR* is empty in non-FIFO mode
     - When the transmitter FIFO is empty in FIFO mode with Programmable THRE interrupt mode disabled
     - When the transmitter FIFO is at or below the programmed threshold with Programmable THRE interrupt mode enabled
   - ➢ The *uart_dma_tx_req* goes inactive under the following conditions:
     - When *THR* is written into the Transmitter Holding Register or transmitter FIFO with Programmable THRE interrupt mode disabled.
     - When the transmitter FIFO is full in the FIFO mode.

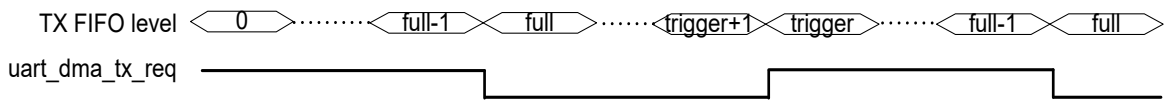


**Figure 14. *uart_dma_tx_req* timing (DMA mode 1)**

2. The uart_dma_rx_req.
   - ➢ The *uart_dma_rx_req* signal becomes active when the following occurs:
     - When the *RBR* is available in non-FIFO mode.
     - When the receiver FIFO is at or above the full trigger level in FIFO mode, or a character timeout has occurred.
   - ➢ The *uart_dma_rx_req* goes inactive under the following conditions:
     - When the *RBR* is read in non-FIFO mode.
     - When the receiver FIFO is empty in the FIFO mode.

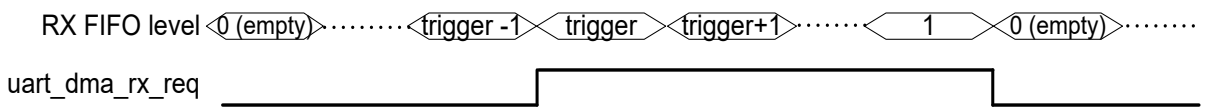


**Figure 15. *uart_dma_rx_req* timing (DMA mode 1)**

### 3.6.2 Single Transaction

- *The uart_dma_tx_single*
  - The *uart_dma_tx_single* signal is a status signal, and is asserted when there is at least one free entry in the transmit FIFO, and cleared when the transmit FIFO is full in the FIFO mode.
  - The *uart_dma_tx_single* signal is a status signal, and is asserted when *THR* is empty, and cleared when writing data into *THR* in the non-FIFO mode.
- The uart _dma_rx_single.
  - The *uart_dma_rx_single* signal is a status signal, and is asserted when there is at least one valid data entry in the receive FIFO, and is cleared when the receive FIFO is empty in the FIFO mode.
  - The *uart_dma_rx_single* signal is a status signal, and is asserted when *RBR* is available, and cleared when reading data from *RBR* in the non-FIFO mode.

**Note:**

The above ouput signals in DMA support can be connected with an interrupt controller as request interrupts.

### 3.6.3 DMA Handshaking

*uart_dma_tx_req, uart_dma_rx_req*–The signals request the *DMAC* to read data from source and write data to destination, *uart_dma_tx_req* and *uart_dma_rx_req*, are activated when their corresponding FIFOs reach the trigger levels. The *DMAC* uses edge detection of the *uart_dma_tx_req*/*uart_dma_rx_req* to identify a request on the channel. Upon reception of the *dma_tx_ack/dma_rx_ack* signal from the DMAC to indicate the burst transaction is complete, the UART APB de-asserts the burst request signals, *uart_dma_tx_req, uart_dma_rx_req*, until dma*_tx_ack/dma_rx_ack is de-asserted* by the *DMAC*.

When the UART APB samples that *_tx_ack/dma_rx_ack is de-asserted* is de-asserted, it can reassert the *uart_dma_tx_req, uart_dma_rx_re* of the request line if their corresponding FIFOs reach the trigger levels. Other, the DMA request lines remain de-asserted.

- *uart_dma_tx_req/ uart_dma_tx_single* handshaking

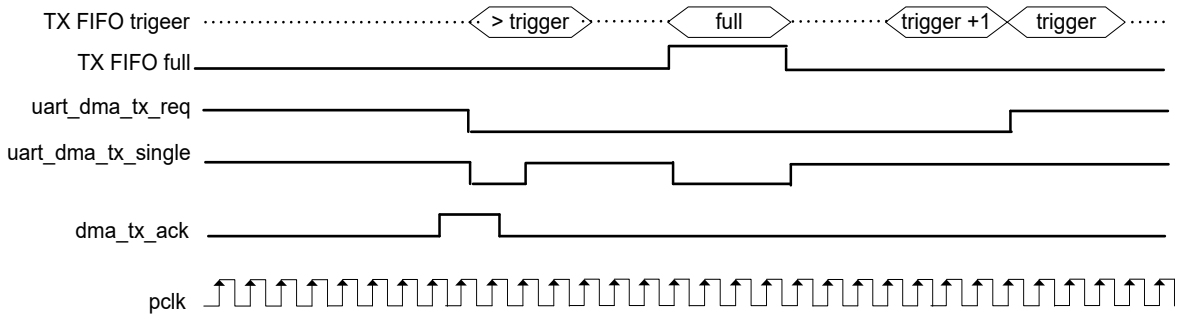


**Figure 16. *uart_dma_tx* timing (a)**

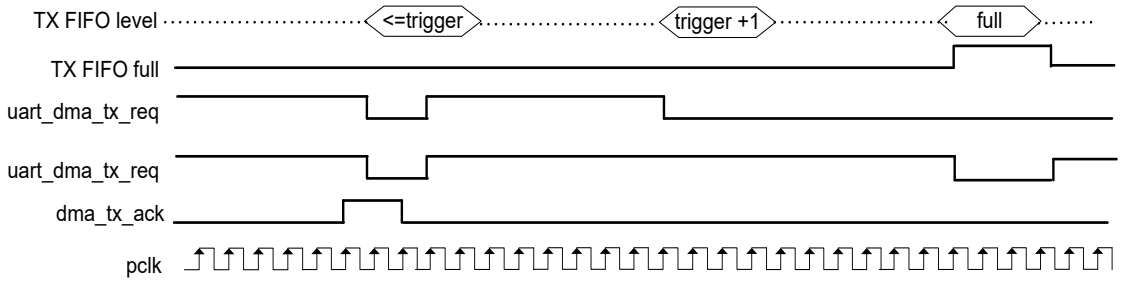


**Figure 17. *uart_dma_tx* timing (b)**

- *uart_dma_rx_req/ uart_dma_rx_single* handshaking

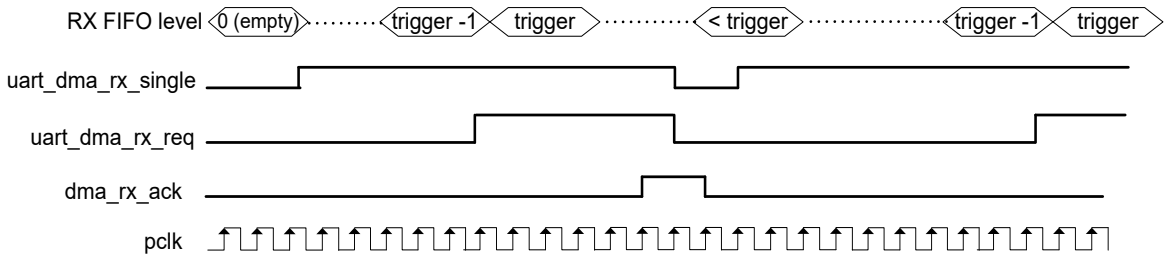


**Figure 18. *uart_dma_rx* timing (b)**

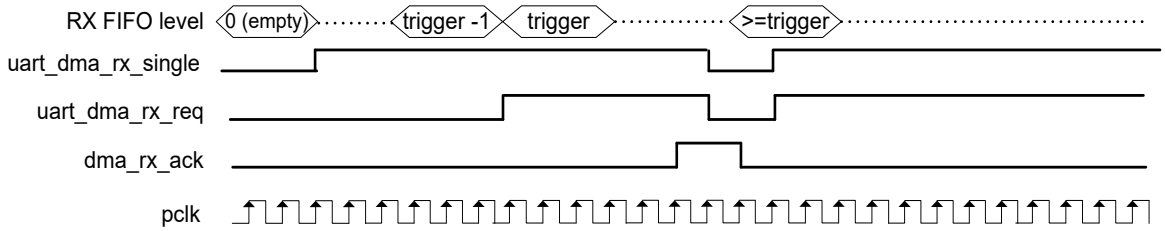


**Figure 19. *uart_dma_rx* timing (b)**

## 4. Parameter Description

**Table 2: Parameter description**

| Name | Description |
|---|---|
| APB_DATA_WIDTH | Width of APB data bus.<br>• Values: 8, 16, 32<br>• Default Value: 32<br>• Dependencies: None |
| FIFO_MODE_USER | Receiver and Transmitter FIFO are selected or not.<br>• Values:<br>✓ No – Non FIFO mode.<br>✓ Yes – FIFO mode.<br>• Default Value: No - Non FIFO mode<br>• Dependencies: None |
| DEEP_FIFO_MODE | Receiver and Transmitter FIFO depth in bytes.<br>• Values: 8, 16, 32,…2048 bytes<br>• Default Value: **8 (change)**<br>• Dependencies: it is only available when FIFO_MODE_USER is selected. |
| MEM_SELECT_USER | Selects between external memory (SRAM) or internal memory (D-FF) for the receiver and transmitter FIFOs.<br>• Values:<br>✓ No - Internal memory.<br>✓ Yes - External memory.<br>• Default Value: No - Internal memory.<br>• Dependencies: It is only available when FIFO_MODE_USER is selected.<br>**Note:** If FIFO depth is greater than ***16-byte***, the external memory should be selected. |
| MODEM_AUTOFLOW_USER | Configures the peripheral to have the 16750-compatible auto flow control mode. For more details, refer to *"auto flow control"*<br>• Values:<br>✓ No - Auto Flow Control is not available. |

| | |
|---|---|
| | ✓ Yes - Auto Flow Control.<br>• Default Value: No - Auto Flow Control is not available.<br>• Dependencies: It is only available when FIFO_MODE_USER is selected. |
| THRE_MODE_USER | Configures the peripheral to have a programmable Transmitter Hold Register Empty (THRE) Interrupt mode. For more information, refer to *"Programmable THRE Interrupt"*.<br>• Values:<br>✓ No - THRE Interrupt mode not available<br>✓ Yes - THRE Interrupt mode<br>• Default Value: No - THRE Interrupt mode not available.<br>• Dependencies: It is only available when FIFO_MODE_USER is selected. |
| DMA_EXTRA_USER | Configures the peripheral to have four additional DMA signals on the interface so that the device is compatible with the DMA controller interface requirements.<br>• Values:<br>✓ No - Additional DMA signals is not included<br>✓ Yes - Additional DMA signals is included<br>• Default Value: No - Additional DMA signals is not included.<br>• Dependencies: None. |
| DMA_POL_USER | Selects the polarity of the DMA interface signals.<br>• Values:<br>✓ No – DMA signals set to active-high.<br>✓ Yes - DMA signals set to active-low.<br>• Default Value: No - DMA signals set to active-high.<br>• Dependencies: It is only available when DMA_EXTRA_USER is selected. |
| SHADOW_USER | It can be used to reduce the software. Configures the peripheral to have seven additional registers that shadow some of the existing register bits that are regularly modified by software.<br>• Values:<br>✓ No – Shadow registers is not included.<br>✓ Yes - Shadow registers is included. |

| | |
|---|---|
| | • Default Value: No - Shadow registers is not included.<br>• Dependencies: None |
| BUSY_DETECT_USER | Busy Detect Indication. An interrupt is set to high when a master tries to write to the Line Control Register while the UART-APB is busy (*USR[0]* is set to one).<br>• Values:<br>✓ No – Busy Detect is not included.<br>✓ Yes - Busy Detect is included.<br>• Default Value: No - Busy Detect is not included.<br>• Dependencies: None |
| BAUD_CLK_USER | Configures the peripheral to have a baud clock reference output (uart_baudout_n) signal on the interface.<br>Values:<br>✓ No - uart_baudout_n signal not included<br>✓ Yes - uart_baudout_n signal included<br>• Default Value: No - uart_baudout_n signal not included<br>• Dependencies: None |

## 5. Register Memory Map

**Table 3: Register Memory Map**

| Name | Address Offset | Width | R/W | Description |
|---|---|---|---|---|
| RBR | 0x00 | 32 bits | R | Receive Buffer Register<br>• Reset Value: 0x0<br>• Dependencies: LCR[7] bit = 0 |
| THR | | 32 bits | W | Transmit Holding Register<br>• Reset Value: 0x0<br>• Dependencies: LCR[7] bit = 0 |
| DLL | | 32 bits | R/W | Divisor Latch (Low)<br>• Reset Value: 0x0<br>• Dependencies: LCR[7] bit = 1 |
| DLH | 0x04 | 32 bits | R/W | Divisor Latch (High)<br>• Reset Value: 0x0<br>• Dependencies: LCR[7] bit = 1 |
| IER | | 32 bits | R/W | Interrupt Enable Register<br>• Reset Value: 0x0<br>• Dependencies: LCR[7] bit = 0 |
| IIR | 0x08 | 32 bits | R | Interrupt Identification Register<br>• Reset Value: 0x01 |
| FCR | | 32 bits | W | FIFO Control Register<br>• Reset Value: 0x0 |
| LCR | 0x0C | 32 bits | R/W | Line Control Register<br>• Reset Value: 0x0 |
| MCR | 0x10 | 32 bits | R/W | Modem Control Register<br>• Reset Value: 0x0 |
| LSR | 0x14 | 32 bits | R | Line Status Register<br>• Reset Value: 0x60 |
| MSR | 0x18 | 32 bits | R | Modem Status Register<br>• Reset Value: 0x0 |

| | | | | |
|---|---|---|---|---|
| *Reserved* | *0x1C-0x78* | *32 bits* | *-* | *Read as zero* |
| USR | 0x7C | 32 bits | R | UART Status Register<br>• Reset Value: 0x6 |
| TFL | 0x80 | See Description (TFL) | R | Transmit FIFO Level, data number in Tx FIFO<br>• Reset Value: 0x0 |
| RFL | 0x84 | See Description (RFL) | R | Receive FIFO Level, data number in Tx FIFO<br>• Reset Value: 0x0 |
| SRR | 0x88 | 32 bits | W | Software Reset Register<br>• Reset Value: 0x0 |
| SRTS | 0x8C | 32 bits | R/W | Shadow Request to Send<br>• Reset Value: 0x0 |
| SBCR | 0x90 | 32 bits | R/W | Shadow Break Control Register<br>• Reset Value: 0x0 |
| SDMAM | 0x94 | 32 bits | R/W | Shadow DMA Mode<br>• Reset Value: 0x0 |
| SFE | 0x98 | 32 bits | R/W | Shadow FIFO Enable<br>• Reset Value: 0x0 |
| SRT | 0x9C | 32 bits | R/W | Shadow RCVR Trigger<br>• Reset Value: 0x0 |
| STET | 0xA0 | 32 bits | R/W | Shadow TX Empty Trigger<br>• Reset Value: 0x0 |
| HTX | 0xA4 | 32 bits | R/W | Halt TX<br>• Reset Value: 0x0 |
| DMASA | 0xA8 | 32 bits | W | DMA Software Acknowledge<br>• Reset Value: 0x0 |
| *Reserved* | *0xAC-0xFC* | *32 bits* | *-* | *Read as zero* |

# 6. Register Description.

## RBR

- Name: Receive Buffer Register
- Size: 32 bits
- Address Offset: 0x00
- Read/write access: read-only

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:8 | Reserved and read as zero | | |
| 7:0 | Receive Buffer Register | R | This register holds the receive data bits.. Bit 0 is the LSB and is the first bit received. The divisor latch access bit (DLAB), *LCR[7]*, must be 0 to read this register.<br>If in non-FIFO mode or FIFOs are disabled (FCR[0] set to zero), the data in the RBR must be read before the next data arrives, otherwise it is overwritten, resulting in an over-run error.<br>If in FIFO mode and FIFOs are enabled (FCR[0] set to one), if the receive FIFO is full and this register is not read before the next data character arrives, then the data already in the FIFO is preserve, but any incoming data are lost and an overrun error occurs.<br>Reset Value: 0x0 |

## THR

- Name: Transmit Holding Register
- Size: 32 bits
- Address Offset: 0x00
- Read/write access: write-only

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:8 | Reserved and read as zero | | |
| 7:0 | Transmit Holding Register | W | This register holds the data bits to be transmitted. Bit 0 is the LSB and is transmitted first. The divisor latch access bit (DLAB), *LCR[7]*,must be 0 to write to this register.<br>If in non-FIFO mode or FIFOs are disabled and THRE is set, writing a single character to the THR clears the THRE. If writing data to the THR before the THRE is set again, the THR data is overwritten.<br>If in FIFO mode and FIFOs are enabled and THRE is set, x number of characters of data may be written to the THR before the FIFO is full. If writing data to the THR when the FIFO is full, the written data is lost.<br>Reset Value: 0x0 |

## DLH

- Name: Divisor Latch High
- Size: 32 bits
- Address Offset: 0x04
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:8 | Reserved and read as zero | | |
| 7:0 | Divisor Latch (High) | R/W | Upper 8-bits of a 16-bit Divisor Latch register that contains the baud rate divisor for the UART-APB.. The baud rate (BR) clock is generated by dividing the input reference clock as follows:<br>• Baud rate = (PCLK) / (16 * divisor).<br>The divisor latch access bit (DLAB) (bit 7 of LCR) must be 1 to access this register. |

## DLL

- Name: Divisor Latch Low
- Size: 32 bits
- Address Offset: 0x00
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:8 | Reserved and read as zero | | |
| 7:0 | Divisor Latch (Low) | R/W | Lower 8-bits of a 16-bit Divisor Latch register that contains the baud rate divisor for the UART-APB. The baud rate (BR) clock is generated by dividing the input reference clock as follows:<br>• Baud rate = (PCLK) / (16 * divisor).<br>The divisor latch access bit (DLAB) (bit 7 of LCR) must be 1 to access this register. |

## IER

- Name: Interrupt Enable Register
- Size: 32 bits
- Address Offset: 0x04
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:8 | *Reserved and read as zero* | | |
| 7 | PTIME | R/W | Programmable THRE Interrupt Mode Enable that can be written to only when THRE_MODE_USER == Enabled, always readable. This is used to enable/disable the generation of THRE Interrupt.<br>• 0 = disabled<br>• 1 = enabled<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER == None |
| 6:4 | *Reserved and read as zero* | | |
| 3 | EDSSI | R/W | Modem status interrupt enable<br>• 0 = Disabled (default)<br>• 1 = Enabled |
| 2 | ELSI | R/W | Receiver line status interrupt enable<br>• 0 = Disabled (default)<br>• 1 = Enabled |
| 1 | ETBEI | R/W | Transmit holding register empty interrupt enable<br>• 0 = Disabled (default)<br>• 1 = Enabled |
| 0 | ERBFI | R/W | Receive data available interrupt enable<br>• 0 = Disabled (default)<br>• 1 = Enabled |

## IIR

- Name: Interrupt Identity Register
- Size: 32 bits
- Address Offset: 0x08
- Read/write access: read-only

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:8 | *Reserved and read as zero* | | |

| | | | |
|---|---|---|---|
| 7:6 | FIFOE | R | FIFOs Enabled. This is used to indicate whether the FIFOs are enabled or disabled. This bit is available when FIFO_MODE_USER is selected.<br>• 00 = disabled<br>• 11 = enabled<br>• Reset Value: 0x00 |
| 6:4 | *Reserved and read as zero* | | |
| 3:0 | Interrupt ID<br>(or IID | R | Interrupt ID. This indicates the highest priority pending interrupt which can be one of the following types:<br>• 0000 = modem status<br>• 0001 = no interrupt pending<br>• 0010 = THR empty<br>• 0100 = received data available<br>• 0110 = receiver line status<br>• 0111 = busy detect<br>• 1100 = character timeout |

## Interrupt Identification Bit Values

| IR Value | Priority Level | Interrupt Type | Interrupt Source | Interrupt Reset Control |
|---|---|---|---|---|
| 0 0 0 1 | - | None | | |
| 0 1 1 0 | Highest | Receiver line status | Overrun error, parity error, frame error or break interrupt | Reading the line status register |
| 0 1 0 0 | Second | Received data available | Receiver data available (non-FIFO mode or FIFOs disabled) or RCVR FIFO trigger level reached (FIFO mode and FIFOs enabled) | Reading the receiver buffer register (non-FIFO mode or FIFOs disabled) or the FIFO drops below the trigger level (FIFO mode and FIFOs enabled) |
| 1 1 0 0 | Second | Character timeout indication | No characters in or out of the RCVR FIFO during the last 4 character times and there is at least 1 character in it during this time | Reading the receiver buffer register |
| 0 0 1 0 | Third | THR empty | Transmitter holding register empty (Prog. THRE Mode disabled) or XMIT FIFO at or below threshold (Prog. THRE Mode enabled) | Reading the IIR register (if source of interrupt); or, writing into THR (FIFOs or THRE Mode not selected or disabled) or XMIT FIFO above threshold (FIFOs and THRE Mode selected and enabled). |
| 0 0 0 0 | Fourth | Modem status | Clear to send or data set ready or ring indicator or data carrier detect. Note that if auto flow control mode is enabled, a change in CTS (that is, DCTS set) does not cause an interrupt. | Reading the Modem status register |
| 0 1 1 1 | Fifth | Busy | BUSY_DETECT_USER is selected and a master tries to write to the LCR while the | Reading the UART status registe |

| | | | | |
|---|---|---|---|---|
| | | detect indication | UART APB is busy USR[0] is set to one). | |

## FCR

- Name: FIFO Control Register
- Size: 32 bits
- Address Offset: 0x08
- Read/write access: write-only
- This register is only valid when the UART-APB is configured to have FIFOs implemented (FIFO_MODE_USER == Yes)

| Bits | Name | W | Description |
|---|---|---|---|
| 31:8 | *Reserved and read as zero* | | |
| 7:6 | RCVR Trigger (or RT) | W | These bits are used to set the trigger level for the RX FIFO interrupt.<br>• 00 = 1 character in the FIFO<br>• 01 = FIFO ¼ full<br>• 10 = FIFO ½ full<br>• 11 = FIFO 2 less than full<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER == None |
| 5:4 | TX Empty Trigger (or TET) | W | These bits are used to set the trigger level for the TX FIFO interrupt.<br>• 00 = FIFO empty<br>• 01 = 2 characters in the FIFO<br>• 10 = FIFO ¼ full<br>• 11 = FIFO ½ full<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER== None or THRE_MODE_USER == None |
| 3 | DMA Mode (or DMAM) | W | DMA Mode<br>• 0 = mode 0<br>• 1 = mode 1<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER == None or DMA_EXTRA_USER==None. |
| 2 | TX FIFO Reset (or XFIFOR) | W | TX FIFO Reset. This resets the control portion of the transmit FIFO and treats the FIFO as empty. Note that this bit is 'self-clearing'. It is not necessary to clear this bit.<br>Reset Value: 0x0 |

| | | | |
|---|---|---|---|
| | | | Dependencies: Writes have no effect when FIFO_MODE_USER == None |
| 1 | RX FIFO Reset (or RFIFOR) | W | RX FIFO Reset. This resets the control portion of the receive FIFO and treats the FIFO as empty. Note that this bit is 'self-clearing'. It is not necessary to clear this bit.<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER == None |
| 0 | FIFO Enable (or FIFOE) | W | FIFO Enable. This enables/disables the transmit (XMIT) and receive (RCVR) FIFOs<br>• 0 = disabled<br>• 1 = enabled<br>Reset Value: 0x00<br>Dependencies: Writes have no effect when FIFO_MODE_USER == None |

## LCR

- Name: Line Control Register
- Size: 32 bits
- Address Offset: 0x0C
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:8 | *Reserved and read as zero* | | |
| 7 | DLAB | R/W | Divisor latch access bit. Write is enabled only when UART is not busy (USR[0] is zero). Enables access to the divisor latch registers during read or write operation to address 0 and 1.<br>• 0 = Disabled (default)<br>• 1 = Enabled<br>Reset Value: 0x0 |
| 6 | BC | R/W | Break Control Bit. This is used to cause a break condition to be transmitted to the receiving device. If set to one the serial output is forced to the spacing (logic 0) state. When not in Loopback Mode, as determined by MCR[4], the sout line is forced low until the Break bit is cleared<br>Reset Value: 0x0 |
| 5:4 | EPS | R/W | Even Parity select. Write is enabled only hen UART is not busy (USR[0] is zero)<br>• 00 = parity bit = odd<br>• 01 = parity bit = even<br>• 11 = parity bit = 1<br>• 10 = parity bit = 0<br>Reset Value: 0x0 |
| 3 | PEN | R/W | Parity Enable. Write is enabled only when UART is not busy (USR[0] is zero); otherwise always writable, always readable. This bit is used to enable and disable parity generation and detection in transmitted and received serial character respectively.<br>• 0 = parity disabled<br>• 1 = parity enabled |

| | | | |
|---|---|---|---|
| | | | Reset Value: 0x0 |
| 2 | STOP | R/W | Number of stop bits. Write is enabled only when UART is not busy (USR[0] is zero);<br>• 0 = 1 stop bit<br>• 1 = 1.5 stop bits when DLS (LCR[1:0]) is zero, else 2 stop bit<br>Reset Value: 0x0 |
| 1:0 | DLS | R/W | Data Length Select. Write is enabled only when UART is not busy (USR[0] is zero);<br>• 00 = 5 bits<br>• 01 = 6 bits<br>• 10 = 7 bits<br>• 11 = 8 bits<br>Reset Value: 0x00 |

## MCR

- Name: Modem Control Register
- Size: 32 bits
- Address Offset: 0x10
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:6 | *Reserved and read as zero* | | |
| 5 | AFCE | R/W | Auto Flow Control Enable. When FIFOs are enabled and the Auto Flow Control Enable (AFCE) bit is set.<br>• 0 = Disabled (default)<br>• 1 = Enabled<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER == None or MODEM_AUTOFLOW_USER == None. |
| 4 | LB | R/W | LoopBack Bit. When this bit is set, the serial ouput and modem outputs are held high. While serial data output is looped back to the sin line, internally. This mode is used for test purposes.<br>• 0 = Disabled (default)<br>• 1 = Enabled<br>Reset Value: 0x0 |
| 3 | OUT2 | R/W | Controls the Output2 (*uart_out2_n*) signal. Active low.<br>• 0 = *uart_out2_n* is 1 (default)<br>• 1 = *uart_out2_n* is 0<br>Reset Value: 0x0 |
| 2 | OUT1 | R/W | Controls the Output1 (*uart_out1_n*) signal. Active low. |

| | | | |
|---|---|---|---|
| | | | • 0 = *uart_out1_n* is 1 (default)<br>• 1 = *uart_out1_n* is 0<br>Reset Value: 0x0 |
| 1 | RTS | R/W | Request to Send. This is used to directly control the Request to Send *(uart_rts_n)* output. The Request To Send (*uart_rts_n*) output is used to inform the modem or data set that the UART is ready to exchange data.<br>When Auto RTS Flow Control is not enabled (MCR[5] set to zero), the *uart_rts_n* signal is set low by programming MCR[1] (RTS) to a high.In Auto Flow Control is Enabled and active (MCR[5] set to one) and FIFOs enable (FCR[0] set to one), the *uart_rts_n* output is controlled in the same way, but is also gated with the receiver FIFO threshold trigger (*uart_rts_n* is inactive high when above the threshold). The rts_n signal is de-asserted when MCR[1] is set low.<br>Note that in Loopback mode (MCR[4] set to one), the *uart_rts_n* output is held inactive high while the value of this location is internally looped back to an input.<br>Reset Value: 0x0 |
| 0 | DTR | R/W | Data Terminal Ready (*uart _dtr_n*) signal. Active low.<br>0 = *uart _dtr_n* is 1 (default)<br>1 = *uart _dtr_n* is 0<br>Reset Value: 0x0 |

## LSR

- Name: Line Status Register
- Size: 32 bits
- Address Offset: 0x14
- Read/write access: read-only

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:6 | *Reserved and read as zero* | | |
| 7 | FIER | R | This bit is set when there is at least one parity error, framing error, or break indication in the FIFO. (This bit is available when FIFO_MODE_USER is selected).<br>This bit is cleared when the LSR is read and the character with the error is at the top of the receiver FIFO and there are no subsequent errors in the FIFO.<br>Reset Value: 0x0 |
| 6 | TEMT | R | Transmit empty. This bit is set to 1 when both the transmitter FIFO (or THR) and shift registers are empty<br>Reset Value: 0x1 |
| 5 | THRE | R | Transmit Holding Register Empty bit.<br>If THRE mode is disabled (IER[7] set to zero) and regardless of FIFO's being<br>Implemented /enabled or not, this bit indicates that the *THR* or TX FIFO is empty.<br>If THRE mode is enabled (IER[7] set to zero) and FIFO mode is enabled, this bit is set when TX FIFO is at or below trigger level.<br>Reset Value: 0x1 |
| 4 | BI | R | Break interrupt. Indicates that the receive data is at 0 longer than a full word transmission time (start bit +data bits + parity + stop bits). Reading the LSR clears |

| | | | |
|---|---|---|---|
| | | | the BI bit<br>Reset Value: 0x0 |
| 3 | FE | R | Framing error. Indicates that the receive byte did not have a valid stop bit.<br>• 0 = no framing error<br>• 1 = framing error<br>Reading the LSR clears the FE bit<br>Reset Value: 0x0 |
| 2 | PE | R | Parity error. Indicates that the receive byte had a parity error.<br>• 0 = no parity error<br>• 1 = framing error<br>Reading the LSR clears the PE bit<br>Reset Value: 0x0 |
| 1 | OE | R | Overrun error bit. This is used to indicate the occurrence of an overrun error. This occurs if a new data character was received before the previous data was read.<br>In the non-FIFO mode, the OE bit is set when a new character arrives in the receiver before the previous character was read from the RBR. When this happens, the data in the RBR is overwritten.<br>In the FIFO mode, an overrun error occurs when the FIFO is full and a new character arrives at the receiver. The data in the FIFO is retained and the data in the receive shift register is lost.<br>• 0 = no overrun error<br>• 1 = overrun error<br>Reading the LSR clears the OE bit.<br>Reset Value: 0x0 |
| 0 | DR | R | Data Ready bit. This is used to indicate that the receiver contains at least one character in the RBR or the receiver FIFO.<br>• 0 = no data ready<br>• 1 = data ready<br>This bit is cleared when the RBR is read in non-FIFO mode, or when the receiver FIFO is empty, in FIFO mode.<br>Reset Value: 0x0 |

## MSR

- Name: Modem Status Register
- Size: 32 bits
- Address Offset: 0x18
- Read/write access: read-only

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:6 | *Reserved and read as zero* | | |
| 7 | DCD | R | Data Carrier Detect. This is used to indicate the current state of the modem control line dcd_n. This bit is the complement of dcd_n. When the Data |

| | | | |
|---|---|---|---|
| | | | Carrier<br>Detect input (*uart_dcd_n*) is asserted it is an indication that the carrier has been detected by the modem or data set.<br>• 0 = *uart_dcd_n* input is de-asserted (logic 1)<br>• 1 = *uart_dcd_n* input is asserted (logic 0)<br>In Loopback Mode (MCR[4] set to one), DCD is the same as MCR[3] (Out2).<br>Reset Value: 0x0 |
| 6 | RI | R | Ring Indicator. This is used to indicate the current state of the modem control line ri_n. This bit is the complement of *uart_ri_n*. When the Ring Indicator input (*uart_ri_n*) is asserted it is an indication that a telephone ringing signal has been received by the modem or data set.<br>• 0 = *uart_ri_n* input is de-asserted (logic 1)<br>• 1 = *uart_ri_n* input is asserted (logic 0)<br>In Loopback Mode (MCR[4] set to one), RI is the same as MCR[2] (Out1).<br>Reset Value: 0x0 |
| 5 | DSR | R | Data Set Ready. This is used to indicate the current state of the modem control line dsr_n. This bit is the complement of *uart_dsr_n*. When the Data Set Ready input (*uart_dsr_n*) is asserted it is an indication that the modem or data set is ready to establish communications with the DW_apb_uart.<br>• 0 = *uart_dsr_n* input is de-asserted (logic 1)<br>• 1 = *uart_dsr_n* input is asserted (logic 0)<br>In Loopback Mode (MCR[4] set to one), DSR is the same as MCR[0] (DTR).<br>Reset Value: 0x0 |
| 4 | CTS | R | Clear to Send. This is used to indicate the current state of the modem control line *uart_cts_n*. This bit is the complement of *uart_cts_n*. When the Clear to Send input (*uart_cts_n*) is asserted it is an indication that the modem or data set is ready to exchange data with the DW_apb_uart.<br>• 0 = *uart_cts_n* input is de-asserted (logic 1)<br>• 1 = *uart_cts_n* input is asserted (logic 0)<br>In Loopback Mode (MCR[4] = 1), CTS is the same as MCR[1] (RTS).<br>Reset Value: 0x0 |
| 3 | DDCD | R | Delta Data Carrier Detect. This is used to indicate that the modem control line *uart_dcd_n* has changed since the last time the MSR was read.<br>• 0 = no change on *uart_dcd_n* since last read of MSR<br>• 1 = change on *uart_dcd_n* since last read of MSR<br>Reading the MSR clears the DDCD bit. In Loopback Mode (MCR[4] = 1),<br>DDCD reflects changes on MCR[3] (Out2).<br>Reset Value: 0x0 |

| | | | |
|---|---|---|---|
| 2 | TERI | R | Trailing Edge of Ring Indicator. This is used to indicate that a change on the input *uart_ri_n* (from an active-low to an inactive-high state) has occurred since the last time the MSR was read.<br>• 0 = no change on *uart_ri_n* since last read of MSR<br>• 1 = change on *uart_ri_n* since last read of MSR<br>Reading the MSR clears the TERI bit. In Loopback Mode (MCR[4] = 1), TERI reflects when MCR[2] (Out1) has changed state from a high to a low.<br>Reset Value: 0x0 |
| 1 | DDSR | R | Delta Data Set Ready. This is used to indicate that the modem control line *uart_dsr_n* has changed since the last time the MSR was read.<br>• 0 = no change on *uart_dsr_n* since last read of MSR<br>• 1 = change on *uart_dsr_n* since last read of MSR<br>Reading the MSR clears the DDSR bit. In Loopback Mode (MCR[4] = 1), DDSR reflects changes on MCR[0] (DTR).<br>Reset Value: 0x0 |
| 0 | DCTS | R | Delta Clear to Send. This is used to indicate that the modem control line *uart_cts_n* has changed since the last time the MSR was read.<br>• 0 = no change on *uart_cts_n* since last read of MSR<br>• 1 = change on *uart_cts_n* since last read of MSR<br>Reading the MSR clears the DCTS bit. In Loopback Mode (MCR[4] = 1), DCTS reflects changes on MCR[1] (RTS).<br>Reset Value: 0x0 |

## USR

- Name: UART Status Register
- Size: 32 bits
- Address Offset: 0x7C
- Read/write access: read-only

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31:5 | *Reserved and read as zero* | | |
| 4 | RFF | R | Receive FIFO Full. This bit is only valid when FIFO_MODE_USER is selected. This is used to indicate that the receive FIFO is completely full.<br>• 0 = Receive FIFO not full<br>• 1 = Receive FIFO Full<br>This bit is cleared when the RX FIFO is no longer full.<br>Reset Value: 0x0 |
| 3 | RFNE | R | Receive FIFO Not Empty. This bit is only valid when FIFO_MODE_USER is selected. This is used to indicate that the receive FIFO contains one or more |

| | | | |
|---|---|---|---|
| | | | entries.<br>• 0 = Receive FIFO is empty<br>• 1 = Receive FIFO is not empty<br>This bit is cleared when the RX FIFO is empty.<br>Reset Value: 0x0 |
| 2 | TFE | R | Transmit FIFO Empty. This bit is only valid when FIFO_MODE_USER is selected. This is used to indicate that the transmit FIFO is completely empty.<br>• 0 = Transmit FIFO is not empty<br>• 1 = Transmit FIFO is empty<br>This bit is cleared when the TX FIFO is no longer empty.<br>Reset Value: 0x1 |
| 1 | TFNF | R | Transmit FIFO Not Full. This bit is only valid when FIFO_MODE_USER is selected. This is used to indicate that the transmit FIFO in not full.<br>• 0 = Transmit FIFO is full<br>• 1 = Transmit FIFO is not full<br>This bit is cleared when the TX FIFO is full.<br>Reset Value: 0x1 |
| 0 | BUSY | R | UART Busy. This bit indicates that a serial transfer is in progress. This bit is cleared when UART is idle or inactive.<br>• 0 = DW_apb_uart is idle or inactive<br>• 1 = DW_apb_uart is busy (actively transferring data)<br>Reset Value: 0x0 |

## TFL

- Name: Transmit FIFO Level
- Size: *log2(FD) + 1*
- Address Offset: 0x80
- Read/write access: read-only

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: log2(FD) +1 | *Reserved and read as zero* | | |
| log2(FD)<br>FD : fifo depth | Transmit FIFO Level | R | Transmit FIFO Level. This is indicates the number of data entries in the transmit FIFO.<br>Reset Value: 0x0 |

## RFL

- Name: Receive FIFO Level
- Size: *log2(FD) + 1*
- Address Offset: 0x80
- Read/write access: read-only

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: log2(FD) +1 | *Reserved and read as zero* | | |
| log2(FD)<br>FD : fifo depth | Receive FIFO Level | R | Transmit FIFO Level. This is indicates the number of data entries in the receive FIFO.<br>Reset Value: 0x0 |

## SRR

- Name: Software Reset Register
- Size: 32 bits
- Address Offset: 0x88
- Read/write access: write-only

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: 3 | *Reserved and read as zero* | | |
| 2 | XFR | W | XMIT FIFO Reset. This is a shadow register for the XMIT FIFO Reset bit (FCR[2]).<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER== None. |
| 1 | RFR | W | RCVR FIFO Reset. This is a shadow register for the RCVR FIFO Reset bit (FCR[1]).<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER== None. |
| 0 | UR | W | UART Reset.<br>Reset Value: 0x0 |

## SRTS

- Name: Shadow Request to Send
- Size: 32 bits
- Address Offset: 0x8C
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: 1 | *Reserved and read as zero* | | |
| 0 | Shadow Request to Send | R/W | Shadow Request to Send Bit. This is a shadow register for the Break bit.<br>Reset Value: 0x0 |

## SBCR

- Name: Shadow Break Control Register
- Size: 32 bits
- Address Offset: 0x90
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: 1 | *Reserved and read as zero* | | |
| 0 | Shadow Break Control Register | R/W | Shadow Break Control Bit. This is a shadow register for the Break bit (LCR[6]).<br>In Loopback Mode, as determined by MCR[4], the sout line is forced low until the Break bit is cleared.<br>Reset Value: 0x0 |

## SDMAM

- Name: Shadow DMA Mode
- Size: 32 bits
- Address Offset: 0x94
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: 1 | *Reserved and read as zero* | | |
| 0 | Shadow DMA Mode | R/W | Shadow DMA Mode. This is a shadow register for the DMA mode bit (FCR[3]).<br>• 0 = mode 0<br>• 1 = mode 1<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when DMA_EXTRA_USER == None. |

## SFE

- Name: Shadow FIFO Enable
- Size: 32 bits
- Address Offset: 0x98
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: 1 | *Reserved and read as zero* | | |
| 0 | Shadow FIFO Enable | R/W | Shadow FIFO Enable. This is a shadow register for the FIFO enable bit (FCR[0]). If this bit is set to zero (disabled) after being enabled then both the XMIT and RCVR controller portion of FIFOs are reset.<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER == None. |

## SRT

- Name: Shadow RCVR Trigger
- Size: 32 bits
- Address Offset: 0x9C
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: 2 | *Reserved and read as zero* | | |
| 1:0 | Shadow RCVR Trigger | R/W | Shadow RCVR Trigger. This is a shadow register for the RCVR trigger bits (FCR[7:6]).This is used to select the trigger level in the receiver FIFO at which the Received Data Available Interrupt is generated. It also determines when the dma_rx_req_n signal is asserted when DMA Mode (FCR[3]) = 1.<br>The following trigger levels are supported:<br>• 00 = 1 character in the FIFO<br>• 01 = FIFO ¼ full<br>• 10 = FIFO ½ full<br>• 11 = FIFO 2 less than full<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER == None. |

## STET

- Name: Shadow TX Empty Trigger
- Size: 32 bits
- Address Offset: 0xA0
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: 2 | *Reserved and read as zero* | | |
| 1:0 | Shadow TX Empty Trigger | R/W | Shadow TX Empty Trigger. This is a shadow register for the TX empty<br>trigger bits (FCR[5:4]). This is used to select the empty threshold level at which the THRE Interrupts are generated when the mode is active.<br>The following trigger levels are supported:<br>• 00 = FIFO empty |

| | | | |
|---|---|---|---|
| | | | • 01 = 2 characters in the FIFO<br>• 10 = FIFO ¼ full<br>• 11 = FIFO ½ full<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when THRE_MODE_USER == None. |

## HTX

- Name: Halt TX
- Size: 32 bits
- Address Offset: 0xA4
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: 1 | *Reserved and read as zero* | | |
| 0 | Halt TX | R/W | This register is use to halt transmissions for testing, so that the transmit FIFO can be filled by the master when FIFOs are implemented and enabled.<br>• 0 = Halt TX disabled<br>• 1 = Halt TX enabled<br>Note, if FIFOs are implemented and not enabled, the setting of the halt TX register has no effect on operation.<br>Reset Value: 0x0<br>Dependencies: Writes have no effect when FIFO_MODE_USER == None. |

## DMASA

- Name: DMA Software Acknowledge
- Size: 32 bits
- Address Offset: 0xA8
- Read/write access: read/write

| Bits | Name | R/W | Description |
|---|---|---|---|
| 31: 1 | *Reserved and read as zero* | | |
| 0 | DMA software Acknowledge | W | This register is use to perform a DMA software acknowledge if a transfer needs to be terminated due to an error condition. For example, if the DMA disables the channel, then the UART APB should clear its request. This causes the TX request, TX single, RX request and RX single signals to de-assert. Note that this bit is 'self-clearing'. It is not necessary to clear this bit. |

| | | | |
|---|---|---|---|
| | | | Reset Value: 0x0<br>Dependencies: Writes have no effect when DMA_EXTRA_USER == No. |

# 7. Signal Description

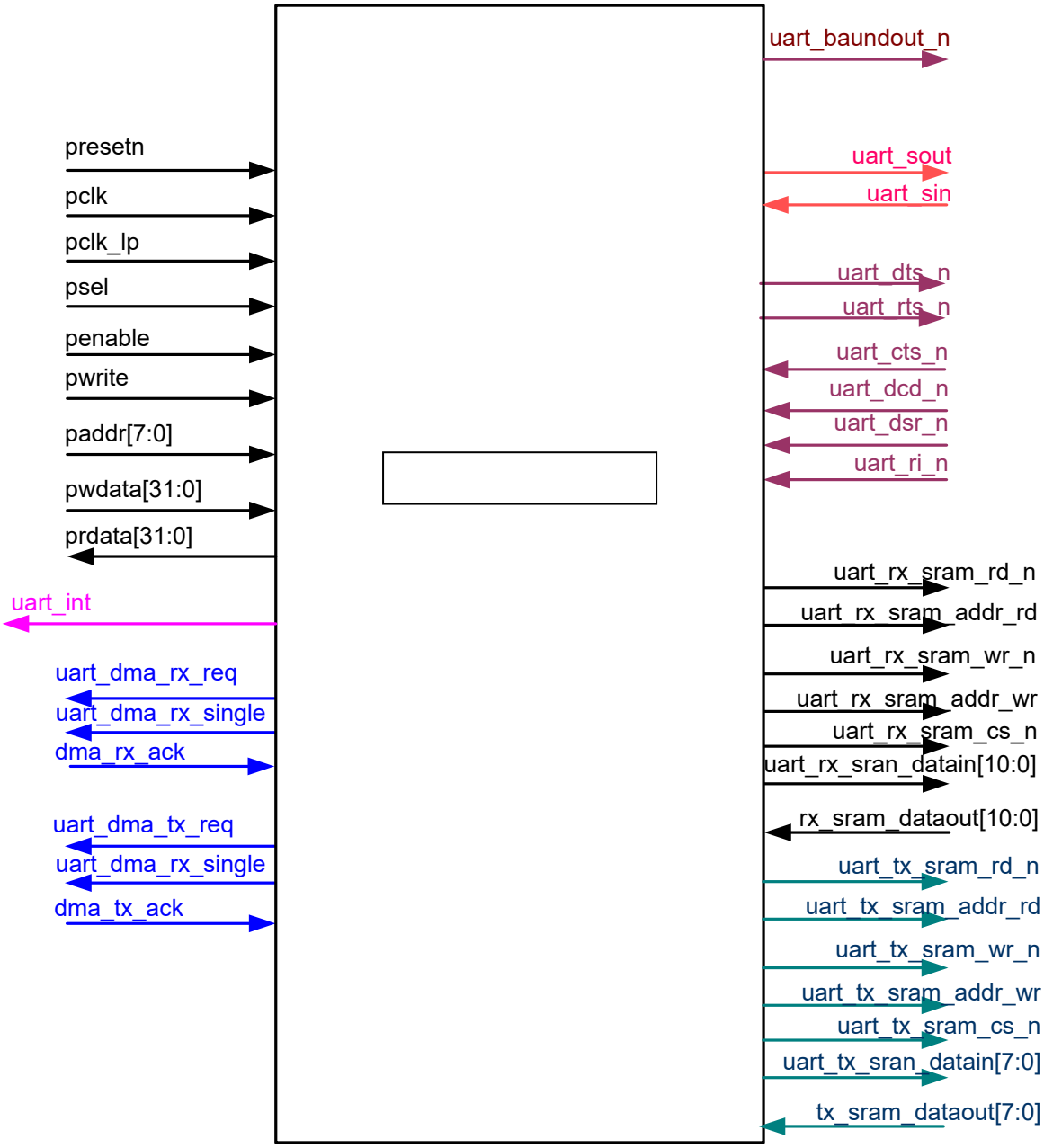


**Figure 20. UART APB signal description**

**Table 4: Signal description**

| Name | Width | I/O | Description |
|---|---|---|---|
| **AMBA PPB** | | | |
| pclk | 1 bit | I | APB clock used in the APB interface to program registers |

| | | | |
|---|---|---|---|
| presetn | 1 bit | I | APB reset |
| psel | 1 bit | I | APB peripheral select |
| paddr | 8 bits | I | APB address bus |
| pwdata | APB_DATA_WIDTH | I | APB write data bus |
| pwrite | 1 bit | I | APB write control |
| penable | 1 bit | I | APB enable control |
| prdata | APB_DATA_WIDTH | O | APB read data bus |
| **INTERRUPT** | | | |
| uart_int | 1bit | O | Uart interrupt flag |
| **FIFO INTERFACE**<br>(FIFO_MODE_USER!=NONE and MEM_SELECT_USER==External) | | | |
| rx_sram_dataout | 11bit | I | APB read data from the receive-FIFO |
| uart_rx_sram_csn | 1bit | I | Data from SRAM<br>Chip enable |
| uart_rx_sram_rd_n | 1bit | O | Read enable |
| uart_rx_sram_add_rd | log2(FIFO_DEPTH) | O | Read address pointer for the receive FIFO SRAM |
| uart_rx_sram_wr_n | 1bit | O | Write enable |
| uart_rx_sram_add_wr | log2(FIFO_DEPTH) | O | Write address pointer for the receive FIFO SRAM |
| uart_rx_sram_datain | 11bit | O | APB read data from the transmit-FIFO |
| tx_sram_dataout | 8bit | I | Data from SRAM |
| uart_tx_sram_csn | 1bit | I | chip enable |
| uart_tx_sram_rd_n | 1bit | O | Read enable |
| uart_tx_sram_add_rd | log2(FIFO_DEPTH) | O | Read address pointer for the transmit FIFO SRAM |
| uart_tx_sram_wr_n | 1bit | O | Write enable |
| uart_tx_sram_add_wr | log2(FIFO_DEPTH) | O | Write address pointer for the transmit FIFO SRAM |
| uart_tx_sram_datain | 8bit | O | Data from Tx SRAM |
| DMA INTERFACE | | | |
| dma_tx_ack | 1 bit | I | Acknowledge from DMA |
| dma_rx_ack | 1 bit | I | Acknowledge from DMA |
| uart_dma_tx_req | 1 bit | O | Transmit Buffer Ready indicates that the Transmit |

| | | | |
|---|---|---|---|
| | | | buffer requires service from the DMA controller. |
| uart_dma_tx_single | 1 bit | O | DMA Transmit FIFO Single informs the DMA controller that there is at least one free entry in the Transmit buffer/FIFO. This output does not request a DMA transfer. |
| uart_dma_rx_req | 1 bit | O | Receive Buffer Ready indicates that the Receive buffer requires service from the DMA controller |
| uart_dma_rx_single | 1 bit | O | DMA Receive FIFO Single informs the DMA controller that there is at least one free entry in the Receive buffer/FIFO. This output does not request a DMA transfer. |
| MODEM INTERFACE | | | |
| uart_cts_n | 1 bit | I | Clear to send modem status |
| uart_dsr_n | 1 bit | I | Data set ready modem status input |
| uart_dcd_n | 1 bit | I | Data carrier detect modem status input |
| uart_ri_n | 1 bit | I | Ring indicator modem status input |
| uart_out1_n | 1 bit | O | Modem Control Programmable output 1 |
| uart _out2_n | 1 bit | O | Modem Control Programmable output 2 |
| uart_dtr_n | 1 bit | O | Modem control data terminal ready output |
| uart _rts_n | 1 bit | O | Modem control request to send output |
| SERIAL INTERFACE | | | |
| uart _sin | 1 bit | I | Serial input |
| uart _out | 1 bit | O | Serial output |
| CLOCK CONTROL INTERFACE | | | |
| uart _baudout_n | 1 bit | O | Transmit clock output |